FireArc R&D Plan — NeuroPhoenix LAB (Sinan Arcana)

Supported by: Technology Division, TTPG

Oct 17, 2026 · @Shamim Reza

Purpose and scope

The technical team turns reference product research into the specifications FireArc is built from. The reference products are Protectli appliances and Firewalla. The outputs are four buildable specs, not reports.

The program covers eight workstreams (A–H below): hardware, boot chain and firmware, the OS base, Firewalla behavior, data plane and IPS, DNS security, NDR and ML, and the agent and response engine. The architecture they feed is set in the FireArc technology and business design document: one Linux (Debian) OS for every model, own hardware built by a Shenzhen ODM, and Firewalla as the feature benchmark.

Deliverable	Owner workstream	Used by
G4 and G6 hardware and firmware specification for ODM quotes	A, B	Procurement, ODM
FireArc OS image: verified boot, A/B update, hardened baseline, reproducible build	C	Platform team
Firewalla parity matrix and behaviour specs (clean-room)	D	Product, all build teams
Data-plane, IPS, DNS and NDR reference configs with benchmark results	E, F, G	Detection and platform teams
Agent protocol and response-engine design with a working prototype	H	Cloud team

Out of scope: building the cloud console, the mobile app or the billing system. Those start once these specs exist.

Ground rules: legal and clean-room

Study how things work, then build FireArc's own versions. Copy nothing. These rules protect the product from license and IP claims later, when partners and investors ask how it was built.

What the team may do

- Buy reference units, take them apart, measure them, and run them on our lab network.

- Read public documentation, datasheets, open-source code, and published standards.
- Observe the behavior of devices we own on our own network: setup flows, alerts, rule effects, performance, and the metadata of their traffic (which hosts they contact, when, how much).
- Read open-source firmware such as coreboot (GPL) and open-source software such as Firewalla's box code (AGPL-3.0), within the clean-room split below.

What the team must not do

- **No copying of code, firmware images, configs, rule content or UI assets** from Protectli, Firewalla or any vendor into FireArc.
- **No dumping or redistributing proprietary firmware** (for example, a vendor AMI BIOS image).
- **No attacks on vendor cloud services:** no attempts to bypass their authentication, break their encryption, or use their APIs beyond what a normal customer account allows. Their cloud is someone else's production system.
- **No use of vendor names or logos** in FireArc material, beyond factual comparison.

Clean-room split for Firewalla

Firewalla's box software is AGPL-3.0. If FireArc code were derived from it, FireArc's box-side source would have to be published. To avoid that:

1. **Study team (Workstream D):** may read Firewalla's code and use the device. It writes behaviour specifications in plain language ("when a new device joins, the owner is alerted within N seconds with its vendor and type") and never includes code or code structure.
2. **Build teams (E–H):** never read Firewalla's source. They build only from the study team's specs and public standards.
3. A reviewer checks each spec before it is handed over. Keep the specs and the review log; they are the evidence of independent development.

Reverse-engineering law differs by country and vendor terms of sale can add limits. Ask counsel in Bangladesh to confirm these rules before Workstreams A and D start.

Lab, reference units and tooling

Set up one isolated lab in Dhaka with its own internet uplink, so reference units and test traffic never touch the office network.

Reference units to buy

Unit	Qty	Why
Protectli Vault Pro VP2440 (N150, 4-port, 2×10G SFP+)	2	G4 reference; one stays sealed and stock, one gets probed and reflashed
Protectli Vault Pro VP6650 (i5, 6-port, 10G SFP+)	2	G6 reference and HA-pair testing
Firewalla Gold SE	1	Feature and UX benchmark (Workstream D only)
Firewalla Purple SE	1	Entry-tier behaviour and limits
ODM samples (from Workstream A shortlist)	3 vendors × 2	Candidates for G4 and G6

Both Protectli models are on Dasharo's supported list, which gives the team an open coreboot+UEFI firmware to study (Dasharo).

Lab network

- An upstream router simulating the ISP (with NAT and CGNAT variants), a managed switch with VLANs and port mirroring, and a victim LAN of Windows, Linux, Android, IP camera and POS-like devices.
- A traffic-generation server with two dual-port 10G NICs: Cisco TRex for stateful mixes, iperf3 for raw throughput, tcpreplay for replaying PCAPs.
- An attack and simulation host for malware PCAP replay, C2 simulation and scan generation. Malware samples stay in an offline, access-controlled vault and never run on lab hosts with internet access.
- A capture host on the mirror port, with Zeek and Wireshark, to record ground truth.

Hardware tools

An SPI flash programmer (CH341A or Dediprog) with SOIC-8 clips, a USB-UART adapter, a logic analyser, a thermal camera, a USB power meter and a bench power supply, an anti-static bench, and a microscope for chip markings.

Shared engineering tools

A Git server with signed commits, CI runners for reproducible image builds, an artifact store, a wiki for specs, and a lab booking calendar.

Workstream A: hardware platform study

Goal: learn exactly what makes a Protectli-class box reliable as a firewall, then write the G4 and G6 board specifications that the Shenzhen ODMs quote against. Owner: 1 hardware engineer + 1 platform engineer. Weeks 1–6.

Tasks

1. **Inventory from software, non-invasively.** On a stock unit, run and archive `lspci -vvnn`, `lsusb -v`, `dmidecode`, `lscpu`, `ethtool -i/-k/-l` per NIC, `sensors`, `nvme id-ctrl`, and `tpm2_getcap properties-fixed`. Record NIC model and queues, PCIe lanes per NIC, TPM type, and storage interface.
2. **Teardown of the probe unit.** Photograph both sides of the board and identify every major chip from its markings: SoC, NIC controllers, Super I/O / EC, SPI flash (size and part), TPM, power regulators, RAM type. Map test points and any UART header.
3. **Thermal and power.** Run CPU and IPS load for 8 hours at 40 °C ambient (a heated enclosure). Log CPU temperature, throttling events and wall power. This sets the fanless thermal budget for Dhaka offices without overnight AC.
4. **NIC behaviour.** Per NIC: number of RSS queues, offloads available, driver (`igc`, `ixgbe`, `ice`), and AF_XDP zero-copy support. Confirm link stability at 2.5G and 10G with long-run iperf3.
5. **Reliability.** Power-cycle each unit 500 times with a relay rig and check that it boots every time. Test brown-out recovery.
6. **Compare ODM samples** against the same tests, side by side.

Output: board specification (G4 and G6)

Item	Requirement
CPU	G4: Intel N150 or N305 class; G6: Core i5/i7 class; AES-NI required
NICs	Intel i226-V for 2.5G; Intel X710 or E810 class for 10G; multi-queue RSS; no Realtek
TPM	Discrete TPM 2.0 (SPI or LPC), not firmware TPM only
SPI flash	At least 16 MB, socketed or with an exposed header for recovery
Firmware	coreboot or open UEFI; Boot Guard not fused, or fused with FireArc's key (see Workstream B)
Console	UART header or USB serial console for recovery
Thermal	Fanless, no throttling at 40 °C ambient under full IPS load
Power	12 V DC, wide-range adapter; recovers cleanly from brown-outs
Availability	3-year component availability commitment and a 6-month end-of-life notice

Workstream B: boot chain and firmware

Goal: FireArc controls every link from power-on to the running OS, and the cloud can verify it. Owner: 1 firmware engineer. Weeks 1–8.

The chain to prove on reference units, then require from the ODM:

1. **Intel Boot Guard / platform root of trust.** Check whether Boot Guard is enabled and fused, and with whose key. A board fused with the ODM's key can never run FireArc-controlled firmware; this is the single most important ODM question. Use public tools to read the Boot Guard and ME status, and record it per sample.
2. **Firmware.** On the probe VP2440, read and back up the vendor SPI image with the hardware programmer, for recovery only and never redistributed. Flash Dasharo (coreboot + UEFI) and confirm the board boots and recovers. Then build Dasharo/coreboot from source and flash a self-built image to prove the team can own the firmware build.
3. **Intel ME/CSME.** Record ME state per model. Dasharo documents soft-disable or HAP-bit disable depending on the Protectli model (Dasharo). Decide the FireArc policy (disabled via HAP where possible) and test that nothing breaks.
4. **UEFI Secure Boot with FireArc keys.** Generate a test PK/KEK/db hierarchy, enroll it, remove the vendor keys, and boot a signed unified kernel image (UKI). Confirm that an unsigned or modified kernel is refused.
5. **Measured boot and attestation.** Confirm PCR 0–7 measurements are stable across reboots and change when firmware or Secure Boot state changes. Prototype remote attestation: the device signs a TPM quote, and a test server checks it against known-good values.

6. **Firmware updates.** Prove that signed firmware updates can be delivered (fwupd/LVFS-style capsule or a FireArc-signed flash tool), with a recovery path if an update fails.
7. **Physical tamper.** Document what an attacker with the box in hand can do (UART, SPI clip, USB boot) and the mitigations: firmware setup password, USB boot disabled, an SPI write-protect strategy.

Outputs:

- A firmware requirements section for the ODM specification.
- The FireArc key ceremony design (offline root, HSM-held signing keys, two-person release).
- A reproducible firmware build in CI.

Workstream C: FireArc OS base

Goal: a minimal, image-based Debian that boots verified, updates atomically and can be rebuilt bit-for-bit. Owner: 2 platform engineers. Weeks 2–10.

Area	What to build and prove	Done when
Image build	Debian stable minimal image with mkosi or debos, pinned package snapshot	Two CI builds of the same commit produce identical hashes
Kernel	Debian LTS kernel with a FireArc config: unused drivers and filesystems removed, lockdown mode on	Boots on all reference units; module list documented
Boot	UKI signed with FireArc test keys; systemd-boot	An unsigned UKI is refused (with Workstream B)
Root filesystem	Read-only root with dm-verity	A modified root block fails to boot and falls back
Updates	A/B slots with RAUC (or Mender), health check, automatic rollback	A deliberately broken update rolls back without a site visit
Data partition	LUKS2 sealed to the TPM (systemd-cryptenroll)	Disk moved to another box cannot be read
Hardening	AppArmor per service, seccomp for parsers, no shell for service users, auditd, CIS Debian benchmark	Benchmark score recorded each build; exceptions justified
Recovery	Factory-reset image; serial console; local break-glass with a TPM-bound one-time code	Field reset tested on every model
SBOM and licences	CycloneDX SBOM per image; licence scanner; GPL source offer generated	SBOM published with every build; no licence blockers

Time budget for boot and update: the box should reach forwarding within 60 seconds of power-on, and an update should cause under 2 minutes of downtime. These are targets to measure, not results.

Workstream D: Firewalla feature and behaviour study

Goal: a complete, clean-room description of what Firewalla does and how it feels to use, so FireArc can match it and go further. Owner: 1 product analyst + 1 network engineer, who form the study team. They do not write FireArc code. Weeks 1–8.

Method

1. **Setup and modes.** Install the Gold SE and Purple SE in each deployment mode the product offers (router, simple/transparent and DHCP-style modes, as available). Record each setup flow screen by screen, how long it takes, and where a non-expert gets stuck.
2. **Feature inventory.** Go through every menu in the app and web portal. For each feature, record what it does, its defaults, its limits, and which tier or box supports it. This becomes the parity matrix.
3. **Behaviour tests.** Drive each feature with lab traffic and record the result: a new device joins; a device contacts a known-bad domain; a large upload; a port scan from inside; a blocked app; a scheduled pause; VPN use; WAN failover. Note what the alert says, how fast it arrives, and what one-tap actions it offers.
4. **Performance.** Throughput and latency with features on and off, using the methodology in the test section, so FireArc has numbers to beat.
5. **Traffic metadata.** On our own network, record which external hosts the boxes contact, how often, and data volumes. This shows what the product depends on in the cloud (updates, intel, relay). Do not decrypt or tamper with the traffic.
6. **Open-source reading (study team only).** Read the AGPL box code (GitHub) to understand concepts, such as how alarms are categorised and which signals feed them. Write the findings as behaviour, never as code or code structure.

Outputs

- **Parity matrix:** each feature, its FireArc equivalent, priority, and the release it ships in.
- **Behaviour specs:** one page per feature, in plain language, with acceptance tests.
- **UX teardown:** annotated flows, with what to copy in spirit and what to improve for South Asian SMEs (languages, partner-managed setup, low-literacy-friendly alerts).
- **Gap list:** what Firewalla does not do that SMEs and partners need, such as multi-tenancy, NDR depth, a SOC handoff and local data.

Workstream E: data plane and IPS

Goal: choose the packet path and IPS mode for G4 and G6 using measurements, not opinions. Owner: 1 network engineer + 1 detection engineer. Weeks 3–10.

Firewall and routing

- nftables with zones, sets and maps, compiled from a policy model (never hand-written rules on the box).
- Flowtables as the fast path for flows that are already established and inspected.
- Multi-WAN with policy routing, SLA probes, and CAKE or fq_codel for queue management.
- WireGuard for site-to-site and remote users; strongSwan for IPsec interop.

Suricata IPS modes to compare

Suricata documents several inline modes on Linux (Suricata docs):

Mode	How it works	Key caveats from the docs	Test on
NFQUEUE	nftables sends packets to Suricata queues; Suricata gives the verdict	Traffic drops if Suricata isn't listening, unless bypass is set (then blocking stops); multi-queue fanout needs one queue per CPU	G4, routed mode
AF_PACKET IPS	Suricata copies packets between two interfaces and drops bad ones	Matching MTU; disable GRO/LRO/TSO; multi-thread load balancing needs eBPF; stream.inline must be on	G4 and G6
DPDK IPS	Kernel bypass on paired NICs	Same thread count per pair; dedicated cores pinned	G6 only, stretch goal

Also evaluate AF_XDP capture as a G6 option on the i226 (igc driver) and E810 (ice driver) NICs, confirming zero-copy support per driver, and Hyperscan for pattern matching.

Rules and tuning

- Start from Emerging Threats Open and design the FireArc rule-pack format: categories, per-tenant enable/disable, suppression lists.
- Profile rule cost and remove or rewrite the 5% of rules that use the most CPU, then re-measure.
- Decide the fail-open versus fail-closed policy per model when Suricata crashes, and test it.

Output

A reference data-plane config per model with benchmark results, the chosen IPS mode, and its failure behaviour.

Workstream F: DNS security and RPZ

Goal: DNS becomes FireArc's cheapest, fastest protection layer and one of its richest NDR signals. Owner: 1 network engineer (shared with E). Weeks 3–8.

Unbound with RPZ

Unbound supports all five RPZ triggers (QNAME, client IP, response IP, NSDNAME, NSIP) since 1.14.0. Its actions include NXDOMAIN, NODATA, PASSTHRU, DROP, TCP-only and local data. Zones load from a file, a URL or AXFR/IXFR, and `rpz-log` with `rpz-log-name` labels each hit by zone (Unbound docs).

Design and prove:

1. **Zone layout.** Separate zones per purpose, evaluated in order: allow-list (PASSTHRU) → FireArc threat intel → partner/tenant custom → category filtering → ads. Use `tags` so different device groups get different zones.
2. **Delivery.** The cloud publishes zones over IXFR or signed HTTPS downloads; measure how long a new indicator takes to reach every box (target: under 5 minutes).
3. **Sinkhole page.** Redirect blocked names to a local block page that explains why, in the user's language, with a "request unblock" button.
4. **Bypass control.** Detect and handle DoH, DoT and hard-coded resolvers: block known DoH endpoints by RPZ and IP sets, intercept port 53 to the local resolver, and alert when a device keeps trying to bypass.
5. **Logging for NDR.** Every query and RPZ hit becomes a structured event with device ID, which feeds DGA scoring and the rare-domain and newly-registered-domain signals in Workstream G.
6. **Scale.** Load-test with 1–5 million RPZ entries on G4 and record memory use, reload time and query latency.

Feeds to evaluate

abuse.ch (URLhaus, ThreatFox), Spamhaus, phishing feeds, newly-registered-domain lists and category lists. For each, record the licence terms for commercial use; many free feeds forbid resale, so a feed may need a commercial licence before it ships.

Workstream G: NDR and machine learning

Goal: behavioural detections that catch what signatures miss, run on the box within its CPU budget, and stay quiet on normal SME traffic. Owner: 1 detection engineer + 1 data scientist. Weeks 3–12.

Sensor and features

- **Zeek** on the box for connection, DNS, TLS, HTTP, SMB and Kerberos logs. Measure its CPU cost per Gbps on G4 and G6. On G2, fall back to Suricata EVE flow and protocol logs.
- **Fingerprints:** JA4 (TLS client), JA4S, HTTP user-agent, DHCP fingerprint. These are used to identify device types and spot odd clients.
- **Feature engine:** a small daemon (Go or Rust) that turns logs into per-device time-window features: destinations, bytes in/out, connection periodicity, DNS entropy, new ports, internal fan-out.

First detections to build

Detection	Signal	Method to start with
C2 beaconing	Regular intervals to a rare destination	Interval-jitter scoring over 1–24 h windows
DGA domains	Random-looking names, many NXDOMAINs	Character n-gram model; NXDOMAIN rate per device
Internal scanning / lateral movement	One device touching many internal hosts or ports	Fan-out thresholds against the device's baseline
Data exfiltration	Upload far above the device's normal, to a new destination	Per-device baseline with robust z-scores
Rare destination	First-seen domain or ASN for the whole site	Site-level frequency table
Ransomware over SMB	Burst of file renames or writes over SMB	Zeek SMB log rate rules
Rogue or new device	Unknown fingerprint on the LAN	Device inventory diff

Start each as a transparent rule or statistic. Add ML (gradient boosting or isolation forest, exported to ONNX and run with ONNX Runtime on the box) only where it beats the simple version on the test set.

Data and evaluation

- **Training and test data:** public labelled datasets for development (for example CIC-IDS2017 and CTU-13), lab-generated attack traffic, and — from the pilot onward — consented, de-identified fleet data.
- **Baseline traffic:** capture at least 2 weeks of normal traffic from 3 friendly SME offices (with written consent) to measure false positives.
- **Metrics:** detection rate per scenario; false alerts per device per week; time to detect; CPU and memory cost on the box.
- **AI analyst inputs:** define the incident record that the cloud LLM receives: detections, evidence and device context, with names and internal IPs tokenised.

Workstream H: agent, adaptive response and cloud interface

Goal: a working `arcd` agent prototype that enrolls a box, applies desired-state config, streams telemetry and executes reversible response actions. Owner: 2 backend engineers. Weeks 4–12.

1. **Enrolment and identity.** The device generates its key in the TPM at provisioning and enrols with a test device CA. Mutual TLS from then on, with no inbound ports on the WAN.

2. **Transport.** Compare gRPC streaming with MQTT over mTLS for reconnection behind CGNAT, bandwidth on weak links, and behaviour after 24 hours offline. Pick one.
3. **Desired-state config.** The cloud sends a signed, versioned policy bundle. The agent validates it, compiles it to nftables, Unbound and Suricata configs, applies it atomically, and rolls back if health checks fail.
4. **Telemetry.** Batched, compressed event streams with local buffering for 72 hours. Measure upload volume per device per day on a typical SME traffic mix, because this drives cloud cost.
5. **Response actions.** Block a destination, quarantine a device to an isolation VLAN, throttle a flow, and undo each one. Every action carries an expiry, the evidence, and who or what approved it.
6. **Guardrails.** A protected-device list, a circuit breaker on action rate, and fail-safe behaviour when the cloud is unreachable.
7. **Remote support.** A time-limited, audited support session brokered by the cloud, off by default.

Output: the agent protocol specification, the response-action catalogue, and a prototype demonstrated end to end on a VP2440.

Test methodology and benchmark targets

Every workstream reports numbers the same way, so results compare across units, modes and weeks.

Traffic mixes

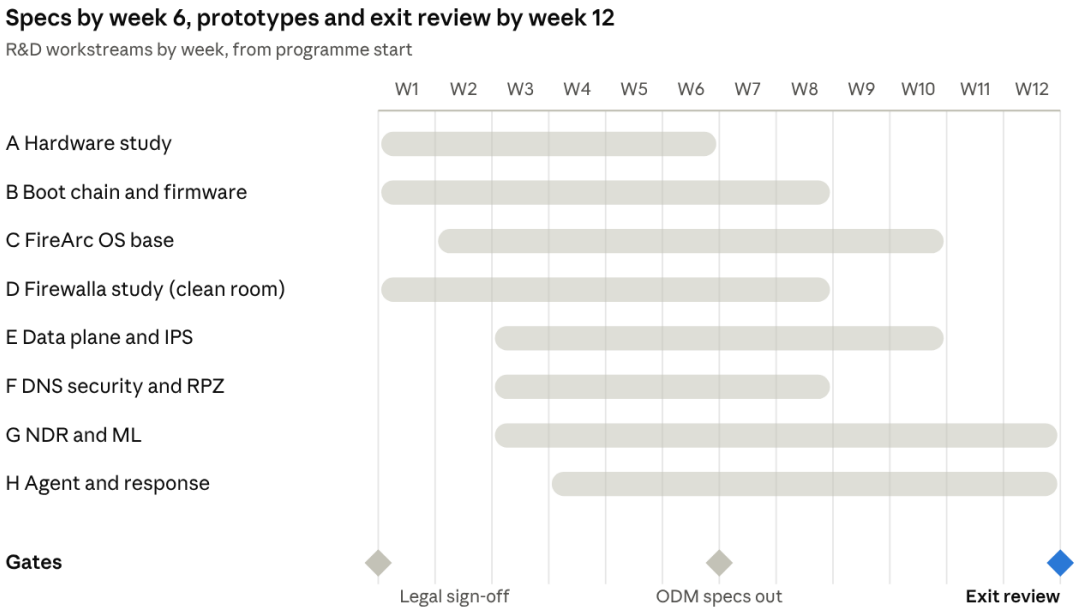
- **SME office mix:** web/TLS-heavy browsing, video calls, cloud storage sync, updates, VoIP and some IoT, replayed with TRex. Build it from the baseline captures in Workstream G.
- **IMIX** for comparison with vendor datasheets.
- **Attack overlays:** malware PCAPs, scans, C2 simulations and DGA lookups, mixed into the office traffic.

Measurements

Metric	How	G4 target	G6 target
Throughput, firewall only	Max rate at under 0.1% loss, SME mix	Measure and record	Measure and record
Throughput, IPS + NDR on	Max rate at under 1% loss, SME mix	1 Gbps	2–5 Gbps
Added latency	p50 and p99 through the box	Under 1 ms p50	Under 1 ms p50
Detection rate	Share of attack-overlay scenarios detected	Record per scenario	Same
False alerts	Alerts per device per week on clean traffic	Under the threshold Product sets	Same
Indicator propagation	New indicator to all boxes	Under 5 min	Under 5 min
Failover (G6 pair)	Session loss during failover	—	No dropped established sessions

Targets are starting goals (throughput from the business plan, the rest set here); the R&D results decide whether they hold. Record every run with the unit, firmware, OS build, ruleset version and traffic file, so any result can be reproduced.

12-week schedule



R&D schedule · 8 workstreams, 3 gates, weeks from programme start

Hardware, firmware and the Firewalla study start first because the ODM specification depends on them. Detection and agent work run to week 12, when the exit review checks every deliverable below. With a team of about 12, people move between workstreams as earlier ones finish.

Exit criteria and deliverables

The R&D phase ends when every item below is done and reviewed. Anything not met becomes a named risk in the build plan, with an owner.

- ☐ Legal sign-off on the ground rules from Bangladeshi counsel (before week 1 work on units)
- ☐ G4 and G6 board specifications sent to 3 Shenzhen ODMs, with Boot Guard and ME status confirmed per sample
- ☐ Self-built firmware boots a reference unit; FireArc-key Secure Boot refuses unsigned kernels
- ☐ TPM quote verified by a test attestation server
- ☐ FireArc OS image: reproducible build, dm-verity root, A/B update with automatic rollback, SBOM
- ☐ Firewalla parity matrix and behavior specs reviewed and handed to build teams (clean-room log kept)
- ☐ IPS mode chosen per model, with benchmark results against the targets
- ☐ RPZ zone design, delivery path and DoH-bypass handling tested; feed licenses checked
- ☐ First seven NDR detections running on the box, with detection and false-alert numbers
- ☐ `arcd` prototype: enroll, apply policy, stream telemetry, execute and undo a quarantine
- ☐ Final R&D report: what worked, what changed in the design, and the updated risk list

References

- FireArc Intelligent Gateway — Technology & Business Design
- Dasharo — Protectli supported models, ME and TPM notes
- Suricata — setting up IPS/inline for Linux
- Unbound — Response Policy Zones
- firewalla/firewalla — box software, AGPL-3.0
- Netgate TNSR — Linux, DPDK and VPP
- 3mdeb — UEFI Secure Boot on FreeBSD with Dasharo (background on firmware signing)